Software Configuration Management / project review

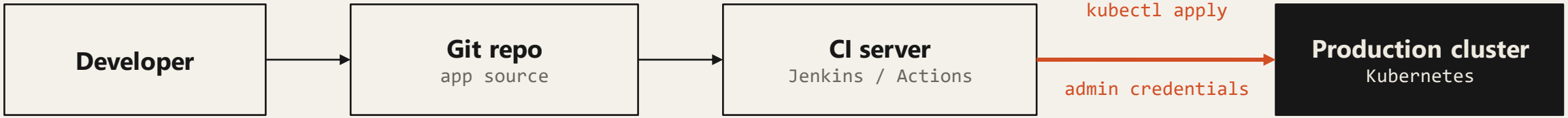
A deployment is a Git commit.

GitOps-based continuous deployment on Kubernetes, using ArgoCD

TEAM

Pallab, Vaibhav

Today, the CI server reaches into production



01

Credentials sit in CI

The CI server stores a production kubeconfig. Anyone who breaks into CI can change the cluster.

02

Configuration drift

An admin runs kubectl edit or scales pods by hand. Git no longer describes what is running.

03

Rollback is a re-run

Going back means running the whole pipeline again for an older commit, and hoping it builds.

GitOps reverses who does the deploying

OPENGITOPS PRINCIPLES

Declarative

The desired state is written in YAML.

Versioned

Every change to that state is a commit.

Pulled

An agent in the cluster fetches changes.

Reconciled

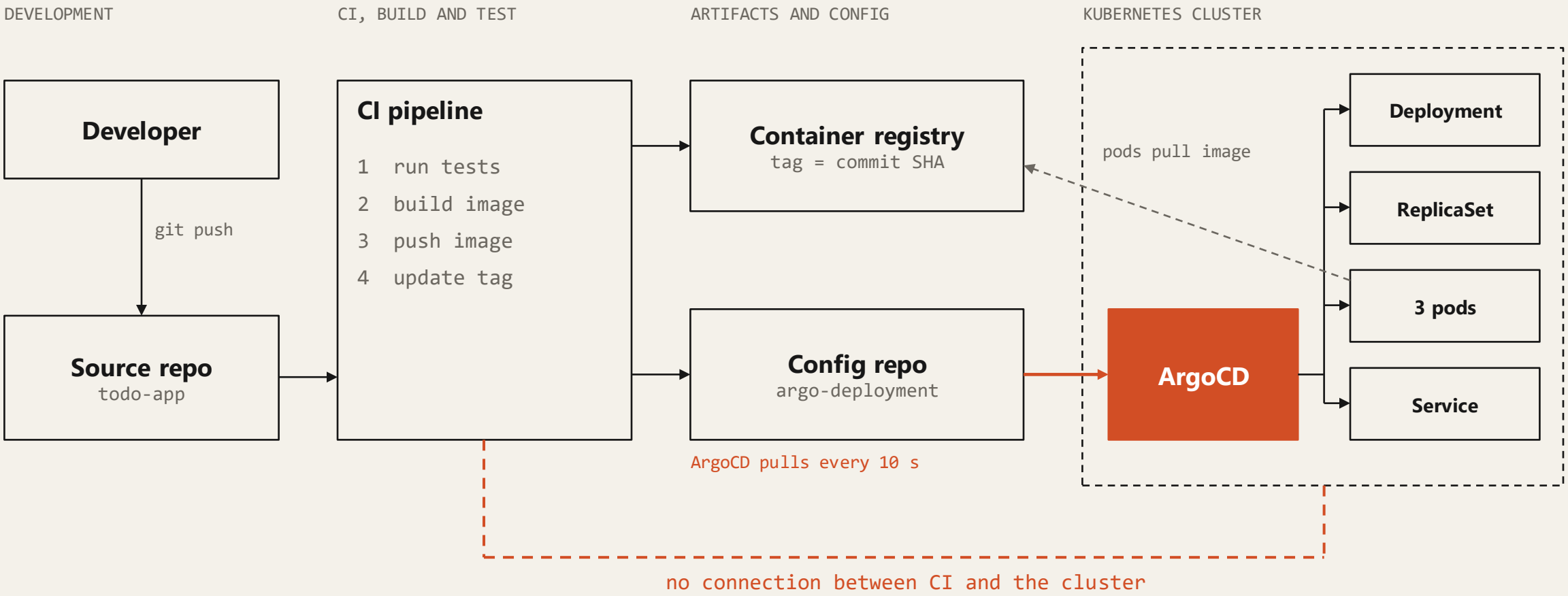
Differences from Git get reverted.

	Push model	Pull model (GitOps)
Who deploys	External CI server	**ArgoCD, inside the cluster**
Cluster credentials	Stored in CI	**Never leave the cluster**
Manual changes	Go unnoticed	**Reverted within seconds**
Rollback	Re-run the pipeline	**`git revert`**
Audit trail	CI job logs	**Git history**

What the project had to prove

1	Single source of truth	All app and cluster configuration lives in Git. Nothing is set by hand.
2	Automated reconciliation	A controller pulls changes into the cluster. No manual deploy step.
3	Zero drift	Manual changes to the live cluster are detected and overwritten.
4	No credentials outside	CI gets Git access only. The deploy mechanism stays behind the cluster boundary.

System architecture



How one change reaches the cluster

CI / push / outside the cluster

CD / pull / inside

STEP 1

Commit

Developer pushes code to todo-app.

STEP 2

Build

Tests run inside docker build. Image tagged with the commit SHA.

STEP 3

Update

CI edits one line, the image tag, in argo-deployment and pushes.

STEP 4

Detect

ArgoCD sees that Git and the cluster differ.

STEP 5

Sync

Rolling update: new pods come up before old ones stop.

`maxUnavailable: 0, maxSurge: 1` keeps 3 pods serving during the rollout.

Every cloud piece, replaced by a local one

ROLE	USUALLY	IN OUR PROJECT
Git hosting	GitHub	**Gitea, self-hosted**
CI	GitHub Actions	**ci-pipeline.sh + prod.yml**
Image registry	Docker Hub	**registry:2 on localhost:5001**
Kubernetes	DigitalOcean	**kind: 1 control plane, 2 workers**
GitOps controller	ArgoCD	**ArgoCD v3.5.3**
Application	Any service	**Node.js todo app, 3 replicas**

```
> .\gitops setup
```

builds all of it in about 5 minutes, then runs with no internet

Two files do most of the work

infra/argocd-application.yaml

```
source:
  repoURL: http://gitea:3000/gitops/
           argo-deployment.git
  targetRevision: main
destination:
  server: https://kubernetes.default.svc
syncPolicy:
  automated:
    prune: true
    selfHeal: true
```

selfHeal re-applies Git when the cluster differs. prune deletes objects removed from Git.

scripts/ci-pipeline.sh (shortened)

```
docker build -t $REGISTRY/todo-app:$SHA .
docker push $REGISTRY/todo-app:$SHA

git clone .../argo-deployment.git ops
sed -i "s|image: .*|image: ...:$SHA|" \
    ops/manifest.yml
git -C ops commit -am "Update tag to $SHA"
git -C ops push origin main

# no kubectl, no kubeconfig
```

The CI script stops at git push. It has no way to reach the cluster.

Five things to watch

1	Deploy a new version with only a git push	`.\gitops release`
2	The CI bot's commit appears in the config repo	`Gitea commit list`
3	Scale to 8 pods and delete the service by hand	`.\gitops drift-demo`
4	Roll back with a git revert	`.\gitops rollback`
5	Push code with a failing test	`.\gitops release broken`

Measured on our laptop

35 to 60 s

git push to all 3 pods on the new version

6 s

for ArgoCD to undo a manual scale and a deleted service

19 s

rollback through git revert

0

kubectl commands run by CI or the developer

OBJECTIVE / EVIDENCE

✓ **Single source of truth**

Every deploy and rollback was a Git commit.

✓ **Automated reconciliation**

ArgoCD synced with no manual step.

✓ **Zero drift**

Extra pods and the deleted service came back.

✓ **No credentials outside**

CI had Git access only.

ArgoCD polling Git every 10 s. A failing unit test stopped the release before any image was built.

What we learned, and what we would add next

LEARNED

Two repos, two histories.

App commits and deploy commits stay apart, so CI commits can't retrigger CI.

Git log is the deploy log.

Who changed what in production, and when, is a git log away.

Declarative YAML can be reviewed.

A change to the cluster is a diff someone can read before it merges.

NEXT

Helm or Kustomize

One chart, separate values for staging and production.

Git webhooks

Sync on push instead of every 10 seconds.

Sealed Secrets

Encrypted secrets that are safe to commit.

Promotion by pull request

Staging to production as a reviewed merge.

Questions?

Git holds the desired state. ArgoCD makes the cluster match it.

Name 1, Name 2, Name 3, Name 4

Source, scripts and manifests: gitops-project/